From POM to Binary: Correlating Dependency Graph Properties with GraalVM Native Image Migration Effort

Sérgio Oliveira
23170981
MSCCLOUD
National College of Ireland

Abstract

Compiling Java applications into Ahead-Of-Time (AOT) native binaries via GraalVM offers significant performance benefits for all environments. However, migrating these applications in some cases remains a challenge due to dependency complexities that often lead to build failures or require extensive manual configurations. Currently, these difficulties are typically discovered only during the intensive compilation phase, resulting in wasted effort and computational costs. Although existing research focuses on post migration performance, little has been done to predict migration feasibility early in the lifecycle. This work proposes an empirical investigation to determine if the structural properties of Maven's dependency graphs can be used to predict the difficulty of native migration early, by leveraging "Shift-Left" concepts and Software Composition Analysis (SCA) principles. The evaluation involves examining the correlation between the metadata extracted from POM files and the practical results obtained from compilation. The objective of the results is to offer developers a simple method to estimate the readiness of their native systems before committing to the migration process.

***Keywords*— Java, GraalVM Native Image, Software Composition Analysis (SCA), Dependency Graph, Shift-Left**

1 Introduction

The Java Virtual Machine (JVM) has traditionally served as the base of software systems written in Java, executing application bytecode via interpretation before employing Just-In-Time (JIT) compilation. However, in the past few years there has been increasing interest in compiling Java applications AOT into native binaries using tools such as GraalVM Native Image. There are great advantages in using native binaries, including faster startup time, lower memory usage and better resource allocation in containerized environments.

Migrating Java applications JVM to native binaries is still a challenge in some cases, because many applications rely on complex libraries or large dependency ecosystems. As a result, the process of converting Java bytecode into a native executable may fail. Therefore, custom configurations are necessary to ensure the successful conversion. However, in practice, these challenges are not usually revealed until efforts are made to actually create the native image. Taking the concepts of "shift left" in the development lifecycle, the sooner a problem is detected, less effort needs to be expended later on, while reducing computational costs and risks to the overall project. As demonstrated in recent research on DevOps methodologies, by integrating critical requirements in the early stages, rather than deferring implementation to later stages, demonstrates both practical and financial viability by strengthening system integrity before the development cycle advances (Nicho *et al.*, 2025).

Java applications are mostly developed using tools like Apache Maven, where dependencies are defined in a POM file. This file creates a dependency graph of the application. However, managing this is a challenge because the Maven ecosystem is huge. A study of 2.2 million Maven dependencies showed that almost every part of a library is used by at least one project (Harrand *et al.*, 2022). This means even a simple project can have many hidden connections, making the 'world' that GraalVM needs to analyze much larger and more complex.

To address this complexity before the build phase begins, techniques are required to map and evaluate these hidden relationships. An approach that aligns with these techniques is the principles of Software

Composition Analysis (SCA), a well-established field used to map third-party components primarily for security and compliance (Kumar *et al.*, 2024). However, their recent analysis of SCA tools highlighted that they still show strongly limited performance under non-perfect conditions and fail to deliver reliable recommendations in complex areas (Shu *et al.*, 2025). Furthermore, while traditional SCA research focuses on vulnerability detection, there is a gap in leveraging these composition insights to predict architectural compatibility for native compilation.

There are some previous works which focused on a range of topics related to native compilation with GraalVM, for example combining both techniques creating a hybrid approach to obtain performance gains, better startup time and compatibility concerns when working with particular frameworks (Pichler *et al.*, 2023). However, little has been done to discover whether the dependency graph of the application can give early signs of potential difficulty or simplicity of migrating that application to a native binary. Specifically, there are no prior empirical studies that analyze the correlation between Maven dependency graphs and the practical issues encountered during native compilation.

This opportunity drives the primary research question addressed by this work:

Research Question:

- **RQ1:** To what extent can the structural properties of Maven dependency graphs provide early indicators of the difficulty of migrating Java applications to native binaries?

To ensure a systematic and objective investigation, this main question is supported by three sub-questions:

- **SRQ1:** Which graph topology metrics (e.g., node centrality, transitive depth, and coupling) best describe the "analysis surface" of a Maven project?
- **SRQ2:** How do these structural graph metrics correlate with the volume of configuration metadata (e.g., reflection, Java Native Interface (JNI), and resource configs) required for a successful native build?
- **SRQ3:** What is the level of precision of an estimation model that uses only static POM metadata to classify the migration difficulty?

To analyze these questions, this work proposes an exploratory and empirical research in which the dependency graphs of multiple applications written in Java will be inspected and compared with the obtained results for compiling them into native binaries using GraalVM Native Image. In this context, the study focus on whether structural properties of the graph-level metrics, such as size and depth, and the composition in terms of dependencies are correlate with practical signs of migration difficulty, such as build fails, build time or memory consumption, or requiring additional configuration.

This research is not expected to produce a predictive model for native migration, but instead takes a step towards determining whether analysis of dependency graphs can serve as an early indicators of how complex a migration will be. Specifically, this paper tries to offer insight into the difficulty of native compilation in relation to dependency structures, which may help developers and researchers evaluate whether their usage of a specific dependency is worth the risk when using native compilation technologies by leveraging the concepts of Shift Left. Moreover, the findings may provide insight that leads to tools or techniques that can help developers better estimate the readiness of their Java applications for native compilation targets.

The remainder of this document is organized into two more main sections. Section 2 provides a critical Literature Review, contrasting the runtime benefits of native binaries against the engineering effort of AOT migration, while identifying the research gap in predictive architectural analysis. Section 3 details the Research Methods and Specifications, outlining the empirical approach of using SCA and dependency graph metrics to evaluate "Native Readiness" through a "Shift-Left" perspective.

2 Literature Review

This section consists of the theoretical basis used to justify this project. It begins with a comparison between JIT and AOT compilation and the implications of the Closed-World Assumption (CWA), then, it explores the complexity of the Maven dependency ecosystem and the principles of SCA. These topics are crucial for estimating the effort needed for native migration by utilizing structural metadata.

2.1 AOT vs. JIT: Performance and Resource Trade-offs

Existing work is focused on the balance between runtime performance and engineering cost of native binaries. While Pichler *et al.* (2024, 2025) argue that AOT compilation is often associated with improved warm-up times. On the other hand, Vergilio et al. Vergilio *et al.* (2023) contrast that JIT environments can achieve better performance in certain long-running workloads. These studies agree on the benefits of the final artifact but disagree on the build time cost and the trade-off between warm-up and peak performance.

In order to bridge these two worlds, Pichler et al. Pichler *et al.* (2023) introduce the concept of Hybrid Execution, achieving speed-ups of up to 1.7x by executing critical functions natively while keeping the rest in managed JIT mode. However, the authors emphasize that the selection of these functions currently requires manual intervention, a challenge shared by Evaldsson and Bardakani (2024), who report the value of the resulting memory efficiency and concerns regarding migration complexity, build size, and compatibility issues.

Furthermore, Pichler et al. Pichler *et al.* (2023) notes that runtime errors can occur if memory conversion fails and highlight the need for mechanisms that can identify potential failures before they occur. This highlights the relationship between technical capabilities and the engineering challenges associated with native compilation, suggesting that the field lacks automated methods to assess migration complexity early in the development lifecycle.

2.2 Technical Constraints and the Closed-World Assumption (CWA)

The transition from JIT to AOT compilation is handle by the CWA. Unlike the traditional JVM, which allows classes to be loaded dynamically at runtime, GraalVM requires that all reachable code paths are identified and analyzed during the build phase. As described by Pichler et al. Pichler *et al.* (2025), this process depends on a reachability analysis that processes application code, libraries, and the JDK together in order to produce a static executable. While this approach enables faster startup time and improved memory efficiency, it also removes the flexibility of loading new types during execution.

However, static analysis is not always able to fully capture the behavior of Java applications, especially when dynamic features are used. According to the GraalVM technical reference (Oracle Labs, 2026a), mechanisms such as Reflection, JNI, and Dynamic Proxies may not be fully visible during static analysis and therefore require additional configuration. These settings are usually provided through structured metadata, which describe code paths that cannot be automatically detected.

This requirement adds more complexity in the build phase. Generally, the usage of indirect or dynamic calls resolved dependencies in an application requires some manual configuration to compile successfully with native compilation. Thus, the effort required to migrate an application to a native binary is influenced not only by its runtime behavior, but also by dependencies and how they interact with these static analysis constraints. This highlights the importance of having an understanding the application structure when assessing the feasibility of native compilation.

2.3 The Build-Time Constrains: Resource Intensity and Baseline Complexity

To understand the structural demands of GraalVM Native Image, it is necessary to establish a baseline of resource consumption during the compilation process. Table 1 presents the metrics from the generation of an executable "Hello World" based on the standard output defined in the GraalVM documentation (Oracle Labs, 2026c).

Table 1: Observed Build Metrics for a GraalVM Native Image for "Hello World" Java Application

Metric	**Value**	**Technical Significance**
Total Build Time	25.5 seconds	Time elapsed for the complete AOT cycle.
Peak RSS	2.14 GiB	Physical memory (RAM) occupied.
CPU Load	32 threads	CPU threads utilized during the process (88.9% parallelism).
Reachable Types	2,140	Total types identified in the analysis phase.
Reachable Methods	8,997	Total methods scanned for the Closed-World.

The results in Table 1 illustrate the complexity associated with AOT compilation. Even though

a "Hello World" is simple, GraalVM found nearly 9,000 reachable methods. This happens because Java libraries are connected. As Harrand *et al.* (2022) demonstrate in the Maven ecosystem that Java Application Programming Interfaces (APIs) are characterized by a long tail of usage where the vast majority of types are interdependent. In the context of GraalVM, this means that even a single call to System.out.println forces the static analyzer to traverse thousands of reachable methods within the java.base module to satisfy the CWA. Therefore, the high "entry cost" observed in Table 1 is a direct reflection of the API interdependency spectrum described by Harrand, where minimal client usage can awaken a significant portion of the underlying platform's code.

In addition, as observed in Table 1, the Peak RSS of 2.14 GiB characterizes the high computational footprint of the native toolchain. These observations highlight the importance of understanding resource consumption during native compilation. Within the scope of this research, these observations provide the necessary contrast to investigate how the addition of third-party dependencies—and the resulting increase in graph nodes and edges—impacts the feasibility and predictability of the build process in constrained environments.

2.4 Software Composition Analysis (SCA) and Dependency Graphs

SCA is widely used for mapping application structures. While Kumar *et al.* (2024) utilizes SCA for vulnerability detection, Shu *et al.* (2025) argue that current tools remain insufficient for addressing real-world architectural complexities. Beyond this traditional focus on security, SCA provides structured dependency data, often formatted as a Software Bill of Materials (SBOM), which details the intricate relationships between software components (Kumar *et al.*, 2024). As these relationships have multiple layers of transitive dependencies, they provide the necessary metadata to model the application's structure as a dependency graph, which potentially reflects the inherent complexity of a system's architecture.

Despite the availability of these structural insights, there is limited discussion in the literature regarding the estimation of technical effort before the migration process begins. In particular, existing studies have not established a clear relationship between structural dependency graph characteristics and the settings imposed by the CWA. This suggests that the complexity of migration is not fully understood due to the role of dependency structures.

2.5 Research Niche and Expected Contribution

The literature review shows three facts:

1. AOT native compilation delivers runtime benefits but imposes constraints that limit dynamic language features.
2. Due to the Closed-World Assumption, even minimal Maven dependencies can pull large portions of the platform into the native-image build graph, substantially increasing build time and memory consumption during compilation.
3. Existing SCA tools provide structured dependency, but were not designed to indicate architectural compatibility with native compilation.

Despite these insights, there are no empirical connections between structural features of Maven dependency and the difficulty of generating GraalVM Native Image.

This research addresses this gap by correlating graph metrics (size, depth, topology) and objective build outcomes (build success/failure and configuration metadata volume).

The expected contribution is a reproducible diagnostic pipeline that enables empirical evaluation of whether the POM file can serve as early indicators of the difficulty of native migration.

3 Research Methods & Specifications

This section describes the methodological approach to investigate the correlation between Maven dependency graphs and the difficulty of GraalVM native compilation. The goal is to determine if metadata available at the planning stage can predict the effort required to reach an operational native binary.

3.1 Research Design

This study adopts a quantitative empirical research design to explore whether the structural properties of Maven dependency graphs can provide early indicators of the difficulty of migrating Java applications to native binaries. The research has an observational and correlational approach where the investigated systems are not altered. Instead, existing software projects will be analyzed in their original form, and their behavior during native compilation.

The methodological approach utilizes principles that are commonly utilized in empirical software engineering, where the systems will be examined to identify measurable relationships between software characteristics and outcomes. In this context, the study focuses on the relationship between dependency graph complexity and migration difficulty, without manual intervention.

The research is divided into four key steps as illustrated in Figure 1. First, the Collector harvests metadata from project repositories in order to create the SBOM. Second, the Graph Processor transforms these descriptors into graph, while the third step is responsible to extract quantitative metrics. Finally, the Reporting & Visualization phase synthesizes the statistical correlations between the metrics and the observed migration difficulty."

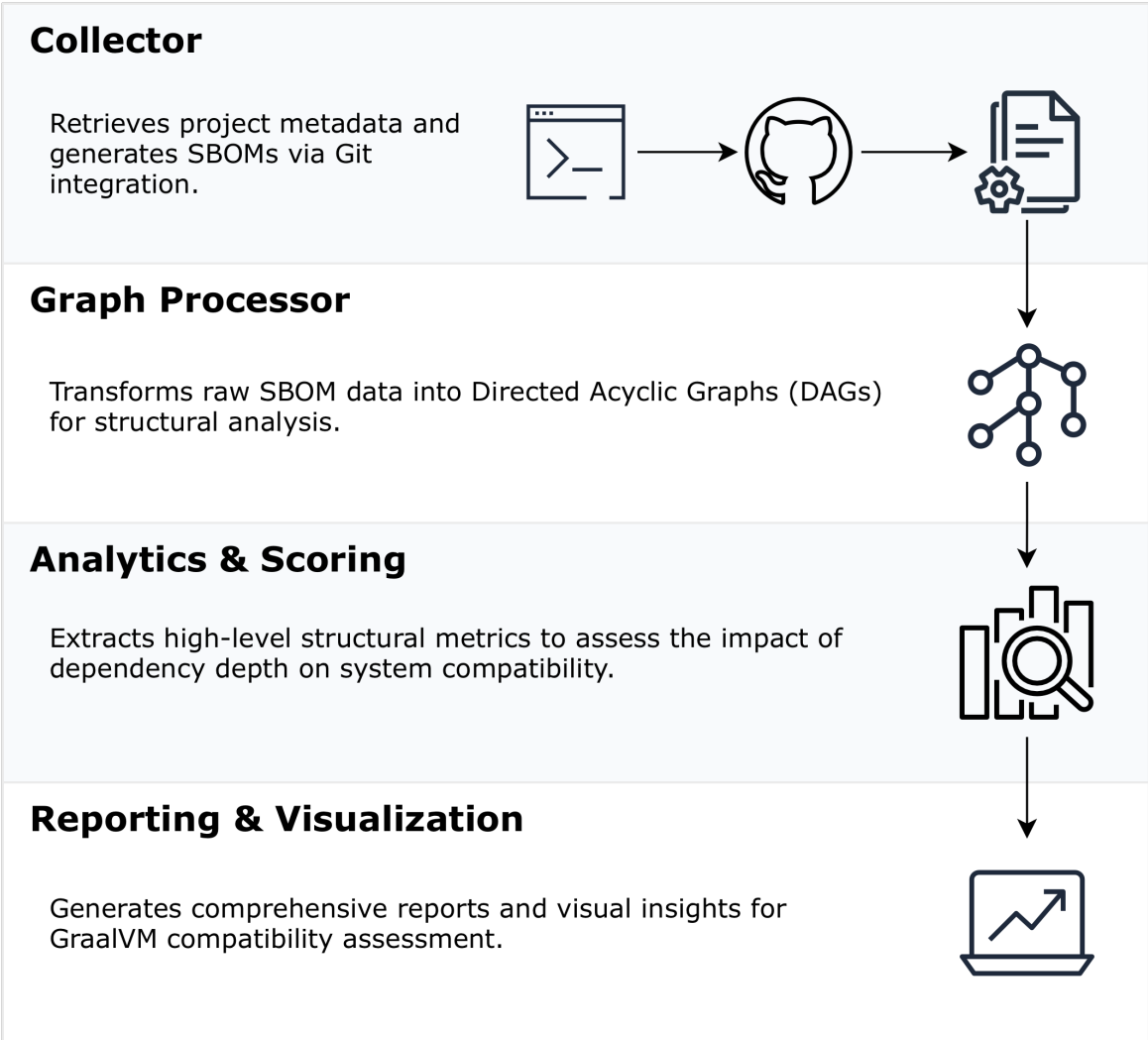

Figure 1: Overview of the research pipeline stages, showing the transformation of raw dependency data into analytical insights.

3.2 Alignment with Research Questions

The methodological design is explicitly aligned with the research questions (see Section 1) defined in this study.

The goal of SRQ1 is to find important metrics for graphical topology and this is done through extraction and computation of structural properties from Maven dependency graphs. These metrics will be utilized to define the complexity and structure of a project's dependencies.

The SRQ2 focus on the investigations of the relationship between graph structure and migration complexity, is addressed through statistical correlation analysis. Specifically, the computed graph metrics are compared against a quantified measure of migration difficulty from build outcomes and logs.

Finally, the interpretation of correlation strength and statistical significance focus on provides an answer to SRQ3 regarding whether it is feasible to estimate migration difficulty based only on static metadata. Instead of building a complete predictive model, the study evaluates if there are any useful predictive signals in dependency graph properties.

Together, these elements support the main RQ1 by determining the extent to which early-stage dependency information can serve as an indicator of native migration difficulty.

3.3 Dataset Selection and Inclusion Criteria

The dataset construction follows a purposive sampling strategy designed to evaluate the impact of structural complexity on GraalVM native compilation. To ensure the reproducibility of the experiment, each candidate project is filtered against a strict set of technical inclusion criteria:

- **Build System:** The project must utilise Apache Maven, ensuring a standardised `pom.xml` structure for metadata extraction.
- **Framework Ecosystem:** Only projects built upon the Spring Boot ecosystem are included to maintain architectural consistency across observations.

The decision to utilise dependency metadata as the primary analytical source is grounded in SCA principles (Kumar *et al.*, 2024). Following the structural dimensions proposed by Shu *et al.* (2025), the research adopts the multilevel dependency topology as the independent variable. Specifically, the dependency count and transitive depth are extracted from each project's dependency tree to characterise its structural profile.

3.4 Variables and Metrics

This study adopts a quantitative approach to investigate the relationship between dependency structure and native compilation outcomes. The variables are defined based on principles from SCA, where dependency metadata is used to derive insights about software systems (Kumar *et al.*, 2024). In addition, graphs are a common abstraction for representing structural complexity in modern applications when modeling dependency relationships (Shu *et al.*, 2025).

Based on this, the variables specified aim to reflect both the complexity of the underlying Maven dependency graph as well as practical configuration constraints that it faces in GraalVM Native build.

3.4.1 Independent Variables

The independent variables consist of structural metrics extracted from the dependency graphs described earlier. These metrics quantify different aspects of dependency complexity within each project.

- **Dependency Count**: The total number of unique dependencies, including both direct and transitive, within the project's SBOM, focus on the overall size of the dependency structure (Massacci and Pashchenko, 2021).
- **Transitive Depth**: The maximum depth of the dependency graph, focus on captures the extent of indirect dependencies within the system (Kumar *et al.*, 2024).
- **Dependency Tree**: The hierarchical structure of the dependency graph used to identify potential structural patterns that may influence native compilation outcomes (Harrand *et al.*, 2022; Kumar *et al.*, 2024; Massacci and Pashchenko, 2021; Shu *et al.*, 2025).

3.4.2 Dependent Variables: Migration Difficulty Index (MDI)

Measuring migration difficulty is a challenge as manual effort can be subjective. To ensure objectivity, this study defines the MDI as a composite proxy composed of two verifiable dimensions.

1. **Migration Difficulty Score (MDS - Build Status):**
   In this study, the dependent variable is the MDS defined as a metric that is indicating whether the GraalVM native compilation process resulted in a success or failure.

   Direct measurement of migration effort would require manual intervention, code modifications, and subjective evaluation, which are outside the scope of this work. The focus of this study is to define migration difficulty as an observable proxy based on build results using a pragmatic approach.

   The build process for GraalVM Native Image requires strict restrictions, as all accessible code must be resolved at compile time and classes cannot be dynamically loaded (Pichler *et al.*, 2025). As a result, additional configuration metadata, such as reflection, JNI, and resource configurations—is often required to ensure successful compilation. These configurations may be defined explicitly or embedded within dependencies and are automatically processed during the build (Oracle Labs, 2026b). Within these constrains, the success or failure of the native build reflects the compatibility between the project's dependency structure and the requirements imposed by compilation.

   The MDS is assigned based on the outcome of the native compilation process using the `native-maven-plugin`, without manual intervention:

   - 0 — Successful;
   - 1 — Failed.

   This classification provides a consistent and reproducible measure of migration difficulty across all projects. This approach avoids subjective bias while enabling systematic comparison between different dependency structures. In addition, it enables the application of non-parametric statistical methods, such as Spearman's rank correlation, to evaluate the relationship between dependency structure and migration difficulty.

2. **Configuration Metadata Volume (CMV):**
   According to Oracle Labs (2026b), dynamic Java features (Reflection, JNI, and Proxies) require explicit metadata to satisfy the CWA. Since the final native image is a composite of all configurations found in all dependencies, the total count of metadata entries will be used as a quantitative measure of effort.

   The CMV is calculated as the sum of:

   - **Reflection Entries:** Count of classes/methods registered for reflective access.
   - **JNI and Proxy Entries:** Number of native interface and dynamic proxy configurations required.

3.5 Experimental Workflow

This study adopts a structured experimental workflow to evaluate how dependency structures relate to native compilation outcome. The procedure is divided into three operational stages.

1. **SCA and Graph Extraction:**

   Each selected project will be processed using the (CycloneDX Project, 2026) tool to generate its SBOM from the `pom.xml`. This step will provide a representation of direct and transitive dependencies, enabling the construction of the corresponding dependency graph.

2. **Native Compilation and Metadata Collection:**

   All projects will be submitted through the GraalVM native compilation process through `native-maven-plugin` with a standardised configuration.

   During this process, two objective outputs are recorded:

   - **Migration Difficulty Score (MDS)**, based on the success or failure of the build;

- **Configuration Metadata Volume (CMV)**, derived from reflection, JNI, and proxy configurations detected during the build.

3. **Data Consolidation:** The structural metrics (Independent Variables) and the migration outcomes (Dependent Variables) will be consolidated into a dataset, mapping all information of each project relating to its observed migration effort.

This approach allows observations to be gained from a controlled, repeatable process that concentrates purely on the structure of each project and the resulting compilation.

3.6 Statistical Analysis Framework

The statistical analysis focuses on the relationship between the structural metrics extracted from dependency graphs and the observed migration outcomes.

To determine relationships between complexity and dependency graph, the Spearman correlation coefficient will be employed.

Each independent variable will also be evaluated individually to determine the direction and strength, and statistical significance will be analyzed to ensure the observed correlations are not due to random variation.

This strategy provides a basis for determining whether structural properties of Maven dependency graphs contain meaningful signals that relate to native compilation difficulty, thereby supporting the investigation of the research question.

3.7 Ethical Considerations

All data used in this study will be obtained from publicly available open source projects. No modifications will be made to the original code, and no private or sensitive data will be retrieved or accessed. The study adheres to standard ethical practices in software engineering research.

3.8 Project Plan

This section presents the timeline that guides this research. The schedule is organized over the 16-week period, highlighting the main phases, tasks, and key milestones that structure the project.

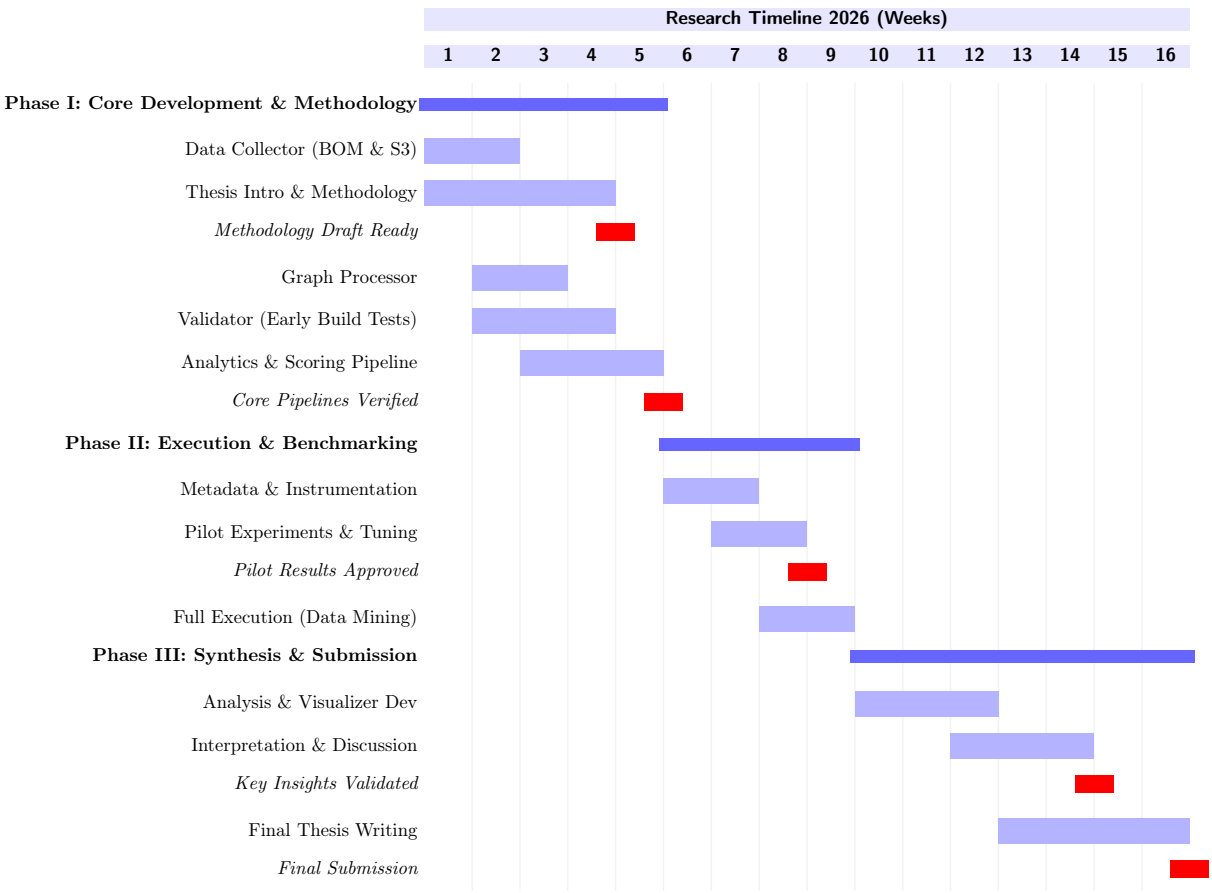

Figure 2: Research Project Timeline

To address environment-level restrictions from the beginning, this project adopts a Hybrid-Iterative methodology, instead of following a traditional waterfall model, where critical issues often appear only in later stages, this approach allows phases to overlap so that critical stages can be validated earlier, preventing errors from propagating into later stages (Leech and Hanslo, 2026; Project Management Institute, 2026).

References

CycloneDX Project (2026). *CycloneDX Python library and SBOM generation tool*. Software version 11.7.0. Available at: https://github.com/CycloneDX/cyclonedx-python-lib [Accessed 18 March 2026].

Evaldsson, T. and M. Y. Bardakani (2024). 'GraalVM's Ahead-of-Time Compilation: Benefits and Challenges in Production'. Bachelor's thesis. Karlskrona, Sweden: Department of Software Engineering, Blekinge Institute of Technology. DiVA id: diva2:1875531. In cooperation with Ericsson.

Harrand, N. *et al.* (2022). 'API beauty is in the eye of the clients: 2.2 million Maven dependencies reveal the spectrum of client-API usages'. In: *Journal of Systems and Software* 184. DOI: 10.1016/j.jss.2021.111134. SJR: 0.950; Quartile: Q1; H-index: 141.

Kumar, S. H. B. I. *et al.* (2024). 'A comprehensive study on the impact of vulnerable dependencies on open-source software'. In: *IEEE International Symposium on Software Reliability Engineering (ISSRE)*. Tsukuba, Japan, 28-31 October 2024, pp. 96–107. DOI: 10.1109/ISSRE62328.2024.00020. CORE Rank: A.

Leech, Bianca and Ridewaan Hanslo (2026). 'The Evolution of Agile and Hybrid Project Management Methodologies: A Systematic Literature Review'. In: *World Symposium on Software Engineering (WSSE)*. Okayama, Japan, 24-26 October 2025, pp. 258–264. DOI: 10.1145/3779657.3779696. Google Scholar citations: 2.

Massacci, F. and I. Pashchenko (2021). 'Technical leverage: Dependencies are a mixed blessing'. In: *IEEE Security & Privacy* 19(3), pp. 58–62. DOI: 10.1109/MSEC.2021.3065627. SJR: 0.643; Quartile: Q1; H-index: 95.

Nicho, M., I. Effiong and C. D. McDermott (2025). 'Shift-left security: Integrating security in the initial phase of the DevOps methodology'. In: *Proceedings of the 2025 9th International Conference on Cryptography, Security and Privacy (CSP)*. Okinawa, Japan: IEEE, pp. 182–191. DOI: 10.1109/CSP66295.2025.00037. Google Scholar citations: 1.

Oracle Labs (2026a). *Native image and third-party libraries*. Available at: https://www.graalvm.org/latest/reference-manual/native-image/native-image-and-third-party-libraries [Accessed 5 March 2026].

— (2026b). *Native image build configuration*. Available at: https://www.graalvm.org/latest/reference-manual/native-image/overview/BuildConfiguration/ [Accessed 20 March 2026].

— (2026c). *Native image build output*. Available at: https://www.graalvm.org/latest/reference-manual/native-image/overview/BuildOutput/ [Accessed 1 March 2026].

Pichler, C. *et al.* (2023). 'Hybrid execution: Combining ahead-of-time and just-in-time compilation'. In: *ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL)*. Cascais, Portugal, 23 October 2023, pp. 39–49. DOI: 10.1145/3623507.3623554. Google Scholar citations: 12.

— (2024). 'On automating hybrid execution of ahead-of-time and just-in-time compiled code'. In: *ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL)*. Pasadena, CA, USA, 20 October 2024, pp. 1–11. DOI: 10.1145/3689490.3690398. Google Scholar citations: 3.

Pichler, C. *et al.* (2025). 'Fast and compact: Reducing size of AOT-compiled Java code without sacrificing performance'. In: *ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR)*. Singapore, 12-18 October 2025, pp. 12–22. DOI: 10.1145/3759426.3760976. Google Scholar citations: 1.

Project Management Institute (2026). *Hybrid Approaches | Hybrid Level 3*. Available at: https://www.pmi.org/disciplined-agile/serial/hybridlifecycles?utm_source=copilot.com [Accessed 20 April 2026].

Shu, C. *et al.* (2025). 'Tool or toy: Are SCA tools ready for challenging scenarios?' In: *Computers & Security* 158. DOI: 10.1016/j.cose.2025.104624. SJR: 1.598; Quartile: Q1; H-index: 148.

Vergilio, T. G., L. D. Ha and A.-L. G. Kor (2023). 'Comparative performance and energy efficiency analysis of JVM variants and GraalVM in Java applications'. In: *International Journal of Environ-*

mental Sustainability and Green Technologies, 14(1), pp. 1–32. DOI: 10.4018/IJESGT.331401. Google Scholar citations: 8.